

Apr'26 Major Examination: EEL3090 Embedded Systems *Set – B*

Guidelines (Total time: 180 minutes, Maximum Marks: 48):

- Please read question paper very carefully. **NO clarification is required in any question.** In case of any doubt, assume and state that in your answer. **Mention Set Number in your answer-sheet.**

1. An electric vehicle uses a CAN-based communication network to exchange real-time information among the Battery Management System, Motor Controller, Brake Control Unit, Dashboard Display, and Temperature Monitoring Unit. The CAN bus operates at **1 Mbps** using standard **11-bit identifiers**. The Battery Management System sends battery voltage and state-of-charge every **30 ms**, the Motor Controller sends motor speed and torque every **8 ms**, the Brake Control Unit sends emergency braking status only when braking is detected, the Temperature Monitoring Unit sends temperature data every **75 ms**, and the Dashboard requests selected data for display every **150 ms**. Design the CAN communication strategy for this system. (a) Explain how message identifiers should be assigned, (b) how bus arbitration will work if two or more nodes try to transmit at the same time, and (c) how event-driven emergency braking messages can be given higher priority than periodic monitoring messages. **[2 + 2 + 2]**
2. You have developed an embedded medical monitoring device running on an RTOS. The system contains five tasks: a high-priority ECG sampling task that must read ADC data every **1.5 ms**, a medium-priority signal-processing task that filters ECG samples every **8 ms**, a wireless communication task that sends patient data every **75 ms**, a low-priority data-logging task that writes samples to flash memory every **400 ms**, and an alarm task that must execute immediately when an abnormal heartbeat is detected. The ADC, flash memory, and communication interface are shared resources, and the low-priority logging task sometimes locks the memory buffer while the high-priority alarm task needs to access the same buffer. (a) Design a scheduling and synchronization strategy for this system. (b) What could go wrong if a medium-priority task preempts a low-priority task while the low-priority task is holding a resource needed by the high-priority alarm task? **[3 + 2]**
3. An embedded system is designed using a microcontroller to monitor an industrial push-button panel with four input switches **SW1–SW4** connected to GPIO pins and two output LEDs **LED1** and **LED2** also controlled through GPIO. The system must operate as follows: when any switch is pressed, the corresponding event must be timestamped using a hardware timer running at **2 MHz**, and the elapsed time since the last switch press must be calculated and displayed. This timestamp data, along with the switch identity and current LED status, must be transmitted over UART to a host PC terminal at a baud rate of **19200 bps** using **8N1** framing. LED1 should toggle every **250 ms** using a timer-generated periodic interrupt, while LED2 should turn ON only when the UART transmission buffer is actively sending data and turn OFF once transmission is complete. If no switch is pressed for more than **12 seconds**, detected using timer overflow or compare logic, the microcontroller must send an idle warning message over UART and drive both LEDs into a slow blinking pattern at **0.5 Hz**. (a) Describe the timer configuration needed to achieve both the **250 ms** LED toggle and the **12-second** idle timeout detection. (b) Discuss whether a single timer with multiple compare registers or two separate timers would be more appropriate, and justify your choice. (c) Explain the overall interrupt priority scheme for this system, identifying potential race conditions between the timer interrupt and UART transmit interrupt, and suggest how they can be resolved. **[2 + 2 + 3]**
4. A microcontroller-based data acquisition system has a CPU clock of **72 MHz** and uses three interrupt sources: a timer interrupt every **0.5 ms**, an ADC interrupt every **125 μ s**, and a UART receive interrupt whenever one byte is received at **230400 bps**. The timer interrupt sub-routine takes **420 CPU cycles**, the UART ISR takes **650 CPU cycles**, and the ADC ISR currently performs filtering inside the ISR and takes **9000 CPU cycles**. Assume one UART byte, including start and stop bits, contains **10 bits**. (a) Determine the time taken by each ISR in microseconds, (b) the interrupt frequency of each source,

and (c) the percentage CPU time consumed by each interrupt. (d) Then decide whether the system can safely handle all interrupts without missing deadlines. How or how not? (e) Explain what will happen if an ADC interrupt occurs while the UART receive interrupt is waiting, and suggest how the ADC ISR should be redesigned to reduce interrupt latency. [1 + 1 + 1 + 2 + 2]

5. A microcontroller based on an ARM Cortex-M processor is running a small embedded control program written partly in ARM assembly. During debugging, the following instruction sequence is observed:

- LDR R0, =0x20001000
- LDR R2, [R0]
- SUB R2, R2, #12
- CMP R2, #40
- STRGT R2, [R0]
- BLE LOW_LIMIT

The memory location 0x20001000 stores a sensor calibration value before execution begins. Explain what each instruction does, how the value in memory changes, how the comparison affects the condition flags, and why the processor may either update the calibration value or branch to LOW_LIMIT. Also discuss what could go wrong if the programmer misunderstands the difference between loading an address and loading the data stored at that address. Hint: STRGT R2, [R0] means store the value of R2 into the memory address stored in R0, only if the previous signed comparison showed “greater than”. [3 + 2]

6. A smart industrial monitoring device is designed using a microcontroller and an RTOS to handle multiple real-time tasks, including sensor data acquisition, motor control, wireless communication, data logging, and fault alarm generation. The sensor task must read temperature and vibration data every **8 ms**, the motor control task must update PWM signals every **4 ms**, the wireless communication task sends system status every **80 ms**, the data logging task stores values in memory every **250 ms**, and the alarm task must respond immediately whenever a critical fault is detected. (a) Design an RTOS-based scheduling strategy for this embedded system. (b) Explain how task priorities should be assigned, (c) which scheduling policy would be suitable, and (d) how periodic and event-driven tasks should be handled. [2 + 2 + 1 + 1]
7. A fighter aircraft uses an embedded safety controller to decide whether the pilot ejection sequence must be initiated during an emergency. The controller receives data from an altitude sensor every **1.5 ms**, an airspeed sensor every **4 ms**, a tilt sensor every **3 ms**, and a pilot ejection handle interrupt that must be detected within **0.8 ms**. After the handle is pulled, the controller must verify aircraft altitude, speed, and attitude, send a command to the seat-control unit, and receive an acknowledgement. The altitude-check task takes **0.35 ms**, the speed-check task takes **0.55 ms**, the command-generation task takes **0.80 ms**, the tilt-check task takes **0.45 ms**, and communication with the seat-control unit takes **1.40 ms**. The total allowed response time from handle pull to confirmed ejection command is **4.5 ms**. (a) Determine whether the embedded controller can meet the deadline. How? (b) Calculate the total processing time and remaining timing margin, and (c) explain what scheduling priority should be given to the pilot-handle interrupt compared with the periodic sensor tasks. Also consider whether a low-priority maintenance task that locks the communication interface for **1.5 ms** can make the system miss its deadline. [2 + 3 + 2]
8. An embedded control unit runs an RTOS on a **168 MHz ARM Cortex-M processor** for a real-time robotic arm. The system has four periodic tasks: **T1**: joint-position sampling every **1 ms** with **100 μ s** execution time, **T2**: motor-control computation every **2.5 ms** with **420 μ s** execution time, **T3**: safety-limit checking every **5 ms** with **650 μ s** execution time, and **T4**: telemetry transmission every **10 ms** with **1.1 ms** execution time. The RTOS uses preemptive priority scheduling, where shorter-period tasks have higher priority. Each context switch requires saving and restoring registers, updating the scheduler state, and reloading the task stack pointer, taking **840 CPU cycles** per switch. It is known that at time $t = 0$, all tasks are released together. (a) Prepare a scheduling analysis for the first **10 ms** and determine whether all task deadlines are met. (b) Calculate the total context-switch overhead in microseconds and CPU utilization due to context switching. [3 + 2]